UNIDAD 5: UTILIZACIÓN DE TÉCNICAS DE PROGRAMACIÓN SEGURA

Módulo Profesional:
Programación de Servicios y Procesos

Índice

RESUMEN INTRODUCTORIO 3
INTRODUCCIÓN 3
CASO INTRODUCTORIO 4
1. CRIPTOGRAFÍA Y SISTEMAS DE IDENTIFICACIÓN 5
1.1 Criptografía. Protocolos criptográficos 5
1.2 Encriptación de información 7
2. MECANISMOS DE SEGURIDAD EN LAS APLICACIONES 10
2.1 Control de acceso. Sistemas de control de acceso basado en roles 10
2.2 Protocolos seguros de comunicaciones 12
2.3 Programación de aplicaciones con comunicaciones seguras 16
RESUMEN FINAL 21

RESUMEN INTRODUCTORIO

En esta unidad introduciremos los conceptos básicos sobre seguridad informática y el desarrollo de software seguro. Para ello, comenzaremos con las definiciones sobre criptografía y criptología dentro de la información y las redes de comunicaciones.

Dentro de las definiciones de seguridad informática relacionada con el desarrollo trabajaremos los conceptos de control de acceso, autenticación y gestión por roles.

Uniremos estos conceptos aplicables a cualquier sistema de la información con nuestro lenguaje base como es Java, estudiando cómo establecer los mecanismos sobre nuestro conocimiento de comunicación mediante sockets para que esta comunicación sea segura y qué protocolos y librerías podemos emplear para la transmisión de datos.

INTRODUCCIÓN

La información es el bien más preciado dentro de una empresa y las aplicaciones que desarrollemos van a ser el medio para trabajar con esta información. Además, esta información es usada por múltiples usuarios y desde múltiples ubicaciones, lo cual es la base de nuestra arquitectura multiusuario y cliente-servidor que hemos visto en unidades anteriores.

La seguridad informática se ha convertido en uno de los elementos más importantes que se debe tener en cuenta en el desarrollo de aplicaciones y por ese motivo, es una de las tendencias más apreciadas en el currículum de cualquier técnico.

Por otro lado, es un estudio muy maduro que parte de estudios matemáticos aplicados al desarrollo de software y que, por lo tanto, partimos de mucho trabajo ya realizado que simplemente debemos implementar en nuestras aplicaciones.

Además, debemos tener en cuenta las comunicaciones entre usuarios, clientes y servidores, y que debemos hacer seguras esas comunicaciones.

También contamos con protocolos ya estándares que simplemente deberemos integrar mediante librerías de Java para envolver tanto nuestra comunicación como nuestra aplicación, y de esa forma hacerla fiable.

CASO INTRODUCTORIO

Somos desarrolladores junior en una cadena de restaurantes y comida para llevar. Se pretende desarrollar una nueva aplicación Java que permita a los clientes realizar pedidos de comida, comprobar el estado de su pedido y, además, a las franquicias revisar las solicitudes.

Nos encontramos en la fase de diseño y planificación de la aplicación y nuestro jefe nos ha encomendado la tarea de encargarnos de la parte de seguridad informática de la aplicación.

¿Qué estándar debemos tener en cuenta para realizar las comunicaciones entre cliente y servidor? ¿Cómo realizamos la autenticación de un usuario? ¿Cómo planificamos la gestión de roles?

Al finalizar la unidad tendremos los recursos y habilidades para:

- Conocer las bases sobre criptografía y seguridad informática.
- Realizar la autenticación de usuarios.
- Realizar la gestión por roles.
- Programar una aplicación y su comunicación segura.

1. CRIPTOGRAFÍA Y SISTEMAS DE IDENTIFICACIÓN

El primer paso es conocer protocolos existentes y formas de comunicación cliente-servidor para intentar adaptar o usar protocolos y servicios ya existentes.

A partir de ese conocimiento, ¿podemos usar un protocolo existente? ¿Tenemos que reinventar uno propio o nuevo?

Antes de comenzar a programar es importante responder a estas preguntas, ya que es importante para plantear los mecanismos de comunicación correctos.

1.1 Criptografía. Protocolos criptográficos

La criptografía es la técnica que permite cifrar mensajes o hacerlos ininteligibles. De esta forma se consigue también proteger estos documentos frente a usos indebidos. Básicamente su funcionamiento consiste en la utilización de códigos para escribir datos confidenciales que circulan por internet o cualquier red en general. Además de lo mencionado, gracias a la criptografía conseguimos:

- Preservar la integridad de la web.
- Autenticar al emisor, receptor del mensaje.
- Confirmar la actualidad del mensaje o el acceso.

Son muchas las aplicaciones que en la actualidad tiene la criptografía:

- **Cifrar**. El cifrado permite que los mensajes que se transmiten no puedan ser conocidos por terceras personas. De esta manera se asegura la privacidad de la información.
- **Autenticar**. El cifrado, además, es utilizado como método de autenticación para demostrar que somos nosotros y que el emisor es quien dice ser. De esta manera es posible cifrar un mensaje con una clave que solo conozcamos nosotros y el receptor podrá confirmar la autoría del mensaje descifrándolo. Para esto se utiliza un sistema de clave simétrica en el que el receptor conoce la clave utilizada para el cifrado.
- **Firmar**. La gran mayoría de trámites que realizamos por internet deben estar firmados digitalmente. Para esto se utiliza un sistema de clave asimétrica en el que se cuentan con dos claves, una privada y

otra pública. El emisor cifra con la clave privada que solo él conoce y el receptor utiliza una clave pública para descifrar.

Los protocolos criptográficos son protocolos que utilizan algoritmos o métodos criptográficos con el objetivo de conseguir algún objetivo en cuestión de seguridad. Por ejemplo, cuando hay algún grado de desconfianza entre el emisor y receptor del mensaje.

Como ejemplo de protocolos podemos citar:

- **SSL**: Protocolo que cifra los mensajes intercambiados entre emisor y receptor a través de un algoritmo de cifrado simétrico y la clave de sesión mediante un algoritmo de cifrado asimétrico. Como ventaja presenta que para cada transacción que se realice genera una clave de sesión distinta por lo que nos protege de que si esta ha sido obtenida por un usuario no autorizado no pueda utilizarla en transacciones posteriores. Este protocolo garantiza la confidencialidad e integridad de los datos intercambiados.

- **SET**: Protocolo utilizado para asegurar las transacciones de internet en las que se necesita tarjeta de crédito. Utiliza dos pares de claves, uno para cifrar la información de la compra y el otro para cifrar la información del pago. Este protocolo garantiza la autenticación del emisor y receptor a través del uso del certificado digital, la confidencialidad de los datos mediante el cifrado y la integridad de la información intercambiada.

- **PGP**: Protocolo que cifra el contenido del mensaje a intercambiar entre emisor y receptor. En este caso utiliza algoritmos de clave simétrica y asimétrica. Garantiza la integridad de la información, la confidencialidad, la autenticación y no repudio.

- **IPSec**: Se trata de un conjunto de protocolos que permiten asegurar las comunicaciones sobre el protocolo IP autenticando y/o cifrando los paquetes IP.

1.2 Encriptación de información

La encriptación de la información consiste en codificar esta de manera que si es interceptada por personas no autorizadas no sean capaces de descifrar y tener acceso a la información. Para descifrar dicha información es necesario una clave que solo deben conocer el emisor y el destinatario de la información.

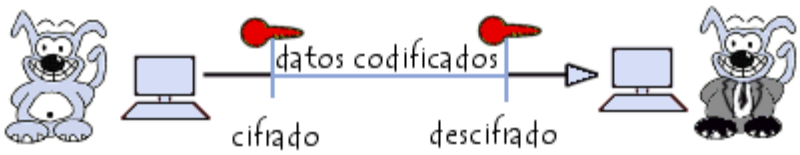
El principal uso de la encriptación está encaminado a proteger la información sensible durante su almacenamiento y transmisión. Nos referimos a información sensible como números de tarjetas de crédito, contraseñas, etc.

Los principales métodos de encriptación son:

- **Criptografía de clave simétrica**

El cifrado de clave privada o simétrica consiste en la utilización de una única clave para el cifrado y descifrado del mensaje enviado.

El funcionamiento consiste en aplicar un algoritmo al mensaje que se va a enviar de manera que queda cifrado. El problema que se puede encontrar en este cifrado es el hecho de que es necesario el envío del mensaje mediante un canal seguro, además de que si un usuario quiere comunicarse con varios usuarios necesita tener tantas claves como usuarios destinatarios de los mensajes.



ENLACE DE INTERÉS

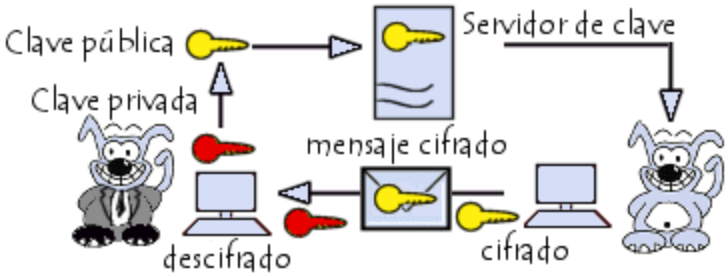
En el siguiente enlace encontrarás más información sobre la criptografía de clave simétrica:

https://www.cert.fnmt.es/curso-de-criptografia/criptografia-de-clave-simetrica

- **Criptografía de clave asimétrica**

El cifrado de clave pública o asimétrica consiste en la utilización de dos claves, una pública para el cifrado y una privada o secreta para el descifrado.

Su funcionamiento es sencillo: un usuario que quiere enviar un mensaje a otro debe cifrar el mensaje que quiere enviar mediante el uso de la clave pública del destinatario del mensaje. Este para conocer el mensaje debe descifrarlo utilizando su clave privada. En la imagen se muestra el proceso:



PARA SABER MÁS

En el siguiente enlace podrás profundizar en tus conocimientos sobre la criptografía de clave asimétrica:

https://www.cert.fnmt.es/curso-de-criptografia/criptografia-de-clave-asimetrica

- **Algoritmo HASH**

El algoritmo HASH es un método que permite generar un número que representa de manera única a un documento o conjunto de datos. Esta generación se realiza a través de una operación matemática. Hay que destacar que a partir del número obtenido no es posible recuperar el conjunto de los datos originales.

Los principales algoritmos HASH son:

- **MD5**: Se trata de una función HASH de 128 bits.
- **SHA-1**: Función HASH de 160 bits.

EJEMPLO PRÁCTICO

La clave de usuario de nuestro cliente la queremos almacenar de forma segura en la base de datos y, para ello, queremos utilizar una codificación MD5 o SHA1. ¿Cómo podemos realizar el proceso de codificación? ¿Cómo podemos crear una clase que nos ayude en esa codificación?

La clase MessageDigest de Java nos permite justamente poder realizar esa codificación de strings (https://docs.oracle.com/javase/7/docs/api/java/security/MessageDigest.html).

Para la realización crearemos la siguiente clase:

```
public class Hash {

    /* Retorna un hash a partir de un tipo y un texto */
    public static String getHash(String txt, String hashType) {
        try {
            java.security.MessageDigest md =
java.security.MessageDigest
                    .getInstance(hashType);
            byte[] array = md.digest(txt.getBytes());
            StringBuffer sb = new StringBuffer();
            for (int i = 0; i < array.length; ++i) {
                sb.append(Integer.toHexString((array[i] & 0xFF) |
0x100)
                        .substring(1, 3));
            }
            return sb.toString();
        } catch (java.security.NoSuchAlgorithmException e) {
            System.out.println(e.getMessage());
        }
        return null;
    }

    /* Retorna un hash MD5 a partir de un texto */
    public static String md5(String txt) {
        return Hash.getHash(txt, "MD5");
    }

    /* Retorna un hash SHA1 a partir de un texto */
    public static String sha1(String txt) {
        return Hash.getHash(txt, "SHA1");
    }

}
```

2. MECANISMOS DE SEGURIDAD EN LAS APLICACIONES

Una vez que ya conocemos los diferentes protocolos y mecanismos, queremos introducir SSL para nuestras comunicaciones en ambas partes de la comunicación.

¿Tenemos librerías en Java que podamos usar para incorporar SSL? ¿Necesitaremos certificados? ¿Necesitaremos implementar mecanismos de autenticación?

Respondiendo a estas preguntas tendremos implementada una aplicación segura extremo a extremos.

2.1 Control de acceso. Sistemas de control de acceso basado en roles

Un sistema de control de acceso basado en roles también conocido como RBAC por sus siglas en inglés consiste en una función de seguridad que permite controlar el acceso de usuarios a determinadas tareas normalmente restringidas.

Estos sistemas proporcionan las siguientes características:

- **Autenticación**: Permite comprobar la identidad del usuario que accede a la aplicación. Este proceso se realiza en dos pasos: primero el usuario se identifica y, a continuación, se comprueba dicha identificación.
- **Autorización**: Permite definir los privilegios de un usuario en una aplicación. Normalmente se sigue una de estas estrategias:
    - Denegar todo desde el principio e ir otorgando permisos según se vaya queriendo.
    - Autorizar todo desde el principio e ir denegando aquellas acciones que no queramos que tenga acceso el usuario.
- **Auditoria**: Permite conservar un historial de la aplicación donde se almacene quién la ha realizado, cuándo, qué permisos tienen los usuarios, etc.

Los sistemas RBAC tienen al menos los siguientes componentes:

- Un repositorio para almacenar la información.
- Un componente integrado en la aplicación para comunicarse con el repositorio.
- Una consola de administración.

- Documentación sobre la aplicación.

Algunos ejemplos de sistemas de control de acceso a usuarios en Java serían:

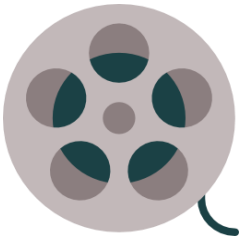
- **Apache Shiro.** Se trata de un framework de seguridad de Java. Se caracteriza por su fácil uso, así como por garantizar la autenticación, autorización, criptografía y gestión de sesiones.

- **Spring Security:** Framework de Java que provee autenticación, autorización y otras características de seguridad para las aplicaciones.

PARA SABER MÁS

En el siguiente enlace encontrarás más información sobre el framework Spring Security:

https://spring.io/projects/spring-security



VIDEO DE INTERÉS

En el siguiente vídeo podrás aprender más sobre el framework Apache Shiro:

https://www.youtube.com/watch?v=YJByiDvOhsc

2.2 Protocolos seguros de comunicaciones

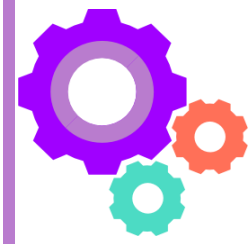
En unidades anteriores hemos estudiado los modelos por capas OSI y TCP/IP. A continuación, vamos a analizar los protocolos seguros que podemos encontrar para las distintas capas:

Nivel de sesión

- **SSL**: Protocolo utilizado para realizar conexiones seguras en internet. Este protocolo cifra los mensajes intercambiados entre emisor y receptor a través de un algoritmo de cifrado simétrico y la clave de sesión mediante un algoritmo de cifrado asimétrico. Como ventaja presenta que para cada transacción que se realice genera una clave de sesión distinta por lo que nos protege de que si esta ha sido obtenida por un usuario no autorizado no pueda utilizarla en transacciones posteriores. Este protocolo garantiza la confidencialidad e integridad de los datos intercambiados.

Cuando enviamos datos a través de la red no podemos estar seguros de que sean accedidos por personas no autorizadas. Esto se agrava cuando la información enviada contiene un número de tarjeta de crédito, una contraseña, etc. Además, no solo es importante que un usuario no autorizado no intercepte el mensaje sino también estar seguros de que este no ha sido manipulado. Para esto se diseñaron la capa de sockets seguros (Secure Socket Layer SSL) y la capa de transporte (TLS), para proteger la privacidad y la integridad de los datos que se transfieren por internet.

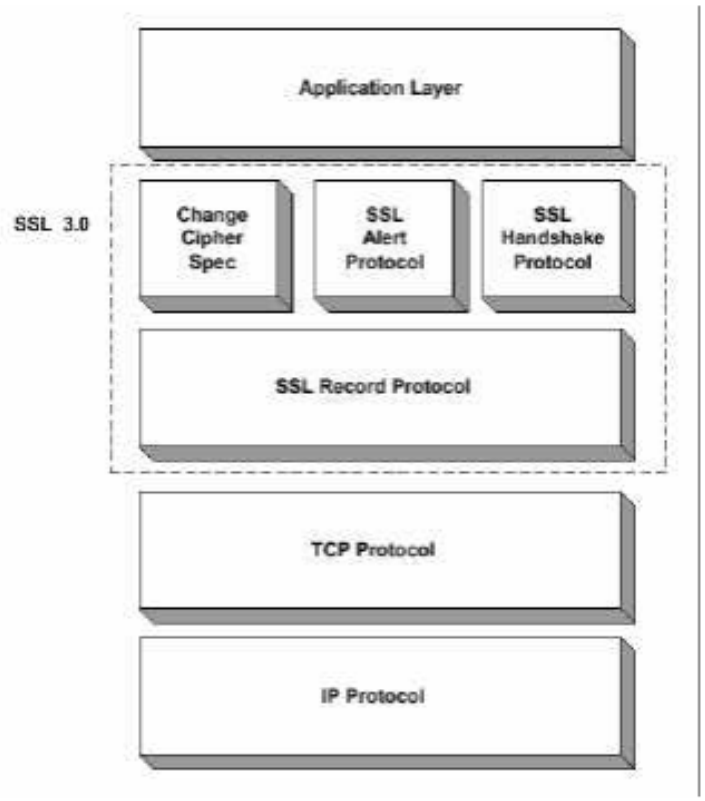
Estas capas proveen mayor seguridad al protocolo estándar TCP/IP utilizado para la comunicación en internet y se colocan entre la capa de transporte y la de aplicación en el caso del modelo TCP o entre la capa de transporte y la de sesión en el modelo OSI.



PARA SABER MÁS

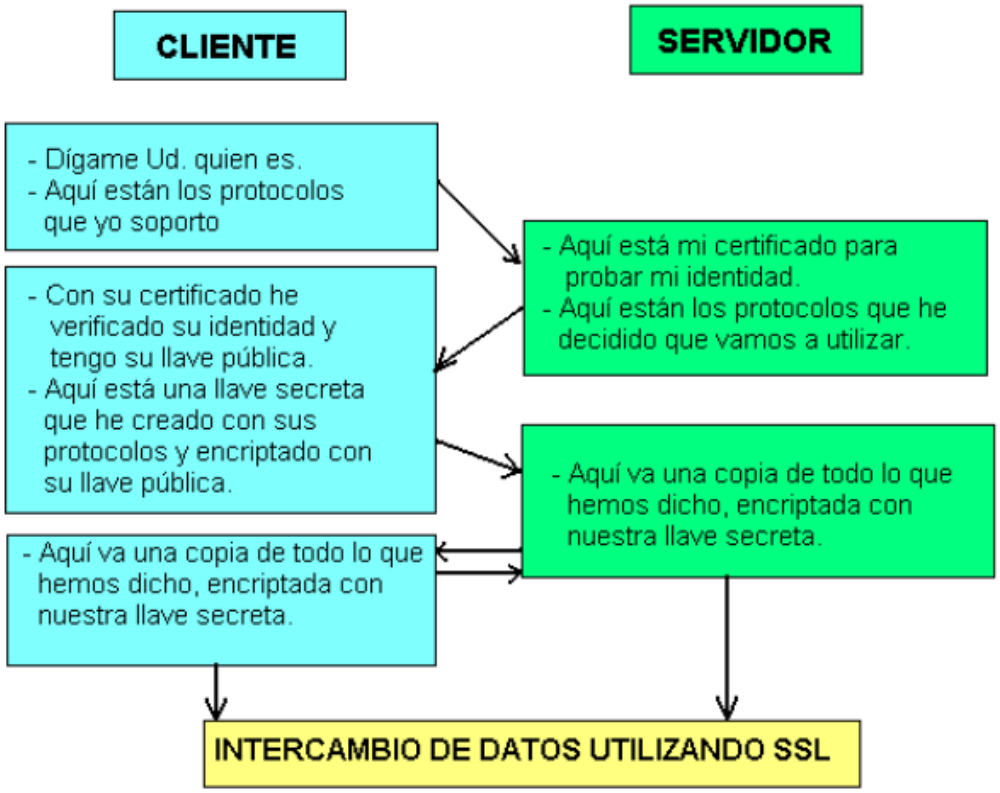
En el siguiente enlace podrás conocer más características sobre el protocolo Secure Socket Layer (SSL):

https://es.ccm.net/contents/141-criptografia-secure-sockets-layers-ssl



Básicamente, SSL funciona de la siguiente manera:

1) Se intercambia una clave mediante algún algoritmo de cifrado asimétrico.
2) A través de esa clave se establece un canal seguro utilizando un algoritmo simétrico.
3) Se fragmenta el mensaje en bloques que se comprimen.
4) Se aplica un algoritmo HASH para obtener un número que se concatena a los bloques del paso anterior para:
   a) Asegurar la integridad de los datos.
   b) Realizar el cifrado.
   c) Enviar los resultados.



Nivel de aplicación

- **HTTPS**: Protocolo seguro de transferencia de datos basado en el protocolo HTTP antes estudiado. Este protocolo crea un canal cifrado basado en SSL/TSL por lo que es más recomendable para usarlo en la transferencia de contraseñas, datos bancarios, etc., que el protocolo HTTP.

ENLACE DE INTERÉS

En el siguiente enlace encontrarás una breve descripción del protocolo HTTPS:

https://www.genbeta.com/web/https-asi-funciona

- **S-HTTP o Secure HTTP**: Protocolo utilizado para realizar transacciones seguras en internet. Para esto utiliza un sistema de cifrado asimétrico.

  Hay que tener cuidado de no confundirlo con el protocolo HTTPS que básicamente se traduce como HTTP sobre SSL. En este caso

garantiza la seguridad del mensaje mediante el protocolo HTTP. Además de:

- Permitir la autenticación de ambos extremos de la comunicación.
- Verificar la integridad de los datos.
- Permitir la negociación de la fortaleza de la clave a utilizar.

La principal diferencia con respecto al protocolo SSL es que S-HTTP realiza transmisiones de mensajes individuales de forma segura. Sin embargo, SSL realiza una conexión segura permanente entre ambas partes. Además, al ser S-HTTP un protocolo de la capa de aplicación proporciona no repudio de mensajes de manera individualizada, así como la capacidad de trabajar conjuntamente con un cortafuegos.

PARA SABER MÁS

En el siguiente enlace podrás encontrar más información sobre el protocolo S-HTTP:

http://www.ecured.cu/index.php/SHTTP

- **S/MIME**: Protocolo de seguridad usado para el intercambio de correo electrónico garantizando la confidencialidad y autoría de los mensajes intercambiados. Al estar basado en el protocolo MIME, nos permite adjuntar cualquier tipo de archivos en nuestros correos electrónicos. Este protocolo permite cifrar el contenido del mensaje mediante algoritmos de clave pública.

2.3 Programación de aplicaciones con comunicaciones seguras

En Java disponemos de las siguientes herramientas de seguridad:

JSSE (Java Secure Sockets Extension). El API de JSSE define un conjunto de clases utilizadas para ejecutar las operaciones necesarias para invocar el protocolo SSL desde el código fuente de Java. Estas clases serán una extensión de las conocidas clases de sockets tradicionales. Esta extensión provee la implementación de los protocolos SSL y TSL para Java además de:

- Integridad del mensaje.
- Encriptación.
- Autenticación del servidor.

PARA SABER MÁS

En el siguiente enlace encontrarás más información sobre JSSE:

https://docs.oracle.com/javase/8/docs/technotes/guides/security/jsse/JSSERefGuide.html

JAAS (Java Authentication and Authorization Service). Proporciona autenticación dentro de la plataforma Java. Todas las facilidades principales del diseño de la seguridad en Java están pensadas para proteger a los usuarios finales de las influencias de los desarrolladores: los usuarios finales dan permiso a los desarrolladores a acceder a los recursos en la máquina del usuario final. JAAS permite a los desarrolladores obtener (o no) el acceso a sus programas basándose en las credenciales de autenticación proporcionadas por el usuario.

ENLACE DE INTERÉS

En el siguiente enlace podrás aprender más sobre JAAS:

https://docs.oracle.com/javase/7/docs/technotes/guides/security/jaas/tutorials/GeneralAcnOnly.html

Vamos a centrarnos en JSSE.

Puesto que el SSL utiliza certificados para la autentificación, necesitamos crear los certificados para nuestro servidor. JSSE puede utilizar los certificados creados por el keytool de Java. De esta forma, la creación de los certificados se hace con la ejecución de keytool, una herramienta de gestión de claves y certificados de Java, con los parámetros pertinentes.

Antes de entrar en materia hay unas características de JSSE que tenemos que conocer.

JSSE distingue entre los keystores y los truststores.

- Keystore, almacén de certificados, (desde la perspectiva de JSSE) es una base de datos de los pares de llaves y de los certificados que se utiliza para la autentificación del SSL.
- En un truststore se almacenan certificados de las Autoridades de Certificación (CA). Se utilizan para verificar las identidades de otros clientes y servidores.

Cuando un cliente o un servidor inician una sesión del SSL, extrae sus certificados y claves de su almacén de certificados (keystore).

Cuando verifica las identidades de otros clientes o servidores, extraerá los certificados de la Autoridad de Certificación en sus truststores. Si se define la variable del sistema javax.net.ssl.trustStore, entonces el valor de esta se utiliza como localización del truststore.

Podemos encontrar el valor de javax.net.ssl.trustStore usando el programa ShowTrustStore.

ARTÍCULO DE INTERÉS

En el siguiente enlace podrás profundizar en los contenidos de keystore y truststore:

https://aprenderdiaadia.wordpress.com/2011/07/23/keystore-truststore/

Los certificados del servidor se pueden generar con un solo comando del keytool. Utilizamos el comando después de crear un certificado de RSA, con el nombre de servidor, y lo salvamos en un keystore llamado certs. Keytool nos pedirá la información que ha de aparecer en el certificado: nombre, organización…

Es imprescindible que cuando keytool nos pida el nombre y apellido del certificado pongamos el nombre del host donde correrá el servidor, en nuestro caso, y puesto que lo utilizamos en un ámbito de prueba escribiremos "localhost".

Para la lectura del fichero que contiene el certificado de clave pública e imprimir el contenido de los campos se utiliza el paquete de java java.security.cert (paquete que añade soporte para generar y usar certificados, así como leerlos de lugares como archivos JAR) y en concreto las clases:

- **certificateFactory**: Utilizada para generar certificados y listas de revocación.
- **X509Certificate**: Clase abstracta utilizada para representar certificados X509. Proporciona un método estándar que se utiliza para acceder a los atributos de un certificado X509.

ARTÍCULO DE INTERÉS

El siguiente artículo trata sobre la seguridad en la plataforma Java:

https://programacion.net/articulo/seguridad_en_la_plataforma_java_2_jdk_1_2_87/11

Ahora ya sí estamos en disposición de conocer cómo construir aplicaciones cliente-servidor SSL.

Antes de nada, vamos a recordar de manera resumida las fases de intercambio de este protocolo:

- En la primera fase el cliente y el servidor negocian la familia de cifradores a utilizar en el intercambio.

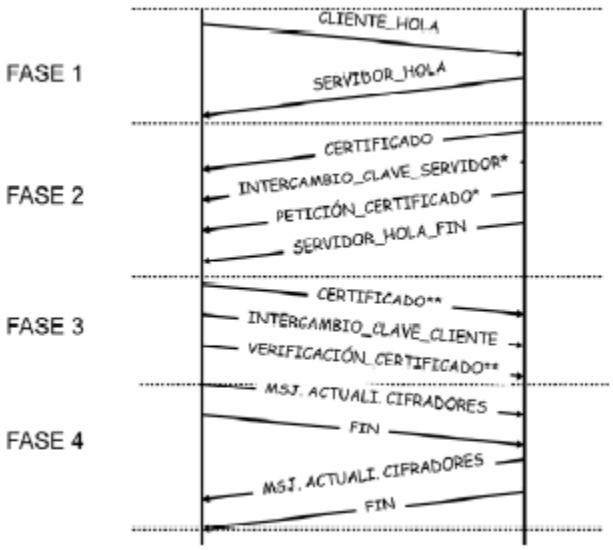
- En la segunda fase el servidor envía un certificado de clave pública con el que autenticarse frente al cliente. El cliente se asegurará que ese certificado recibido es fiable y, además, protege el sitio web al que trata de conectarse el cliente.

Opcionalmente el servidor pude solicitar en esta fase al cliente que envíe un certificado para que se autentifique.

- En la tercera fase el cliente utiliza el certificado de clave pública del servidor para cifrar las claves de sesión con la clave pública del servidor garantizando de este modo la confidencialidad en la distribución de las claves.

  Si el servidor solicitó al cliente autenticación en la fase anterior, el cliente está obligado en esta fase a enviar un certificado de clave pública firmado por una entidad que para el servidor sea fiable. Además, el cliente hará una firma de los mensajes.

- En la última fase se actualizan los cifradores negociados y se envía el primer mensaje cifrado y autenticado con las claves acordadas.



Para que nuestras aplicaciones desarrolladas en Java puedan utilizar el protocolo SSL es necesario que invoquen las clases sockets modificadas:

- **SSL**Socket
- **SSLServer**Socket

Para construir una aplicación SSL lo primero que tenemos que hacer es construir los almacenes de claves necesarios para la ejecución del protocolo SSL. En resumen, en una comunicación SSL típica se tendrán dos ficheros:

- **Fichero keystore**: Contiene el certificado y la clave privada que verifica la identidad del servidor. La clave es utilizada como entrada del algoritmo de intercambio de clave y el certificado se envía al cliente para indicarle quien es el servidor.

Si el servidor pide al cliente autenticación, este debe disponer de su propio keystore con su clave y su certificado.

No hay establecido un keystore por defecto, hay que establecerlo modificando las propiedades del sistema o por código.

- **Fichero truststore**: Contiene solo el certificado. Es utilizado por el cliente para verificar el certificado que le envía el servidor. Se encuentra establecido por defecto.

Ambos almacenes son gestionados por clases que implementan las interfaces **KeyManager** y **TrustManager**.

EJEMPLO PRÁCTICO

Antes de desplegar nuestra aplicación Java que va a usar nuestros clientes para realizar los pedidos, queremos firmarla mediante un certificado digital propio y, de esa forma, cuando se use o se instale en cualquier ordenador, sea de confianza y no de ningún problema. ¿Cómo podemos realizarlo?

Usaremos la utilidad de ORACLE jarsigner (https://docs.oracle.com/javase/7/docs/technotes/tools/windows/jarsigner.html) combinada con la opción keystore. Para ello, usaremos el siguiente comando:

```
jarsigner -keystore empresa.jks -signedjar appPedidosCliente.jar
appPedidos.jar cliente
```

Con esta instrucción:

- Se firma la app appPedidos.jar
- Con la clave privada almacenada con el alias cliente y que se encuentra dentro del fichero empresa.jks
- El fichero una vez firmado se denominará appPedidosCliente.jar

RESUMEN FINAL

Las aplicaciones hoy en día son herramientas informáticas que manejan información por diferentes usuarios y que, además, deben trabajar a partir de una arquitectura cliente-servidor.

Como hemos visto, la seguridad informática y la criptografía nos permite conocer los estándares existentes para realizar las comunicaciones entre dos nodos de forma segura.

Añadida a esa comunicación, vimos que la necesidad de varios usuarios trabajando de forma simultánea hace que nos planteemos desarrollar un sistema de autenticación de usuario, verificando la identidad del mismo, y una vez comprobada esta entrada, realizar una gestión de los usuarios a través de un control de acceso. Asimismo, aprendimos cómo los roles nos permiten estratificar al usuario para acceder y manejar la información por privilegios, lo que crea también otra capa de seguridad en nuestra aplicación.

Por último, hemos unido todos estos conocimientos con nuestro lenguaje base, Java, introduciendo librerías ya implementadas o bien programando nuevo código para incorporarlo a nuestros diseños y software desarrollado con Java.